Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н. Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н. Э. Баумана)

ФАКУЛЬТЕТ «Информатика и системы управления»

КАФЕДРА «Программное обеспечение ЭВМ и информационные технологии»

# ОТЧЕТ

по лабораторной работе № 4

по курсу «Анализ алгоритмов»

Студент ИУ7-51Б (Группа) _____ (Подпись, дата) Платонова М. М. (И. О. Фамилия)

Преподаватель _____ (Подпись, дата) Волкова Л. Л. (И. О. Фамилия)

*2024 г.*

# СОДЕРЖАНИЕ

**ВВЕДЕНИЕ** **3**

**1 Входные и выходные данные** **3**

**2 Преобразование входных данных в выходные** **3**

**3 Примеры работы программы** **4**

**4 Тестирование** **4**

**5 Описание исследования** **5**

5.1 Технические характеристики . . . . . 6

5.2 Результаты исследования . . . . . 6

**ЗАКЛЮЧЕНИЕ** **9**

**СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ** **10**

# ВВЕДЕНИЕ

В современном программировании обработка данных в параллельном режиме является важной задачей, востребованной во многих областях, включая анализ больших данных и автоматизацию процессов. Использование потоков позволяет распределять вычисления и ресурсы для повышения производительности и эффективности программных систем. В данной лабораторной работе нативные потоки рассматриваются для реализации параллельной обработки данных [1].

Целью данной лабораторной работы является освоение навыков организации параллельных вычислений на основе использования нативных потоков.

Для достижения поставленной цели необходимо выполнить следующие задачи:

— исследовать особенности многопточной обработки данных;

— определить входные и выходные данные;

— разработать алгоритмы (один на основе последовательной обработки, второй – с использованием параллельных потоков) для обработки данных;

— провести тестирование разработанных алгоритмов;

— сравнить скорость работы программы при различном количестве потоков.

## 1 Входные и выходные данные

Программа принимает на вход URL кулинарного веб–сайта и количество страниц для загрузки [2]. Результатом работы программы являются HTML файлы с исходной структурой страниц, а также данные о времени выполнения обработки.

## 2 Преобразование входных данных в выходные

Программа получает URL веб–ресурса и извлекает HTML контент с

первой страницы. Загруженный код сохраняется в строковой переменной. После этого выполняется анализ содержимого на наличие ссылок на другие страницы. При нахождении таких ссылок программа осуществляет повторные запросы, извлекая HTML страницы для дальнейшей обработки. Все страницы обрабатываются параллельно, а результаты сохраняются в виде отдельных файлов, которые формируют выходные данные.

## 3 Примеры работы программы

На рисунке 3.1 представлен пример работы программы.

```
marina@MacBook-Pro-Marina ~/D/4/src> /usr/local/bin/python3.10
Меню:
1 - Однопоточный режим
2 - Многопоточный режим
3 - Замеры времени и график производительности
0 - Выход
Выберите опцию (0-3):
```

Рисунок 3.1 – Пример работы программы

## 4 Тестирование

В таблице 4.1 представлены тесты для разработанного класса Scraper, который выполняет однопоточную и многопоточную загрузку страниц с указанного URL.

Таблица 4.1 – Функциональные и производственные тесты для класса Scraper

| № теста | Входные данные | Полученные выходные данные | Ожидаемые выходные данные |
|---|---|---|---|
| 1 | URL `"https://eda.menu"`, 5 страниц, 1 поток | 5 сохраненных HTML файлов | пройден |

Таблица 4.1 – Функциональные и производственные тесты для класса Scraper (продолжение)

| № теста | Входные данные | Полученные выходные данные | Ожидаемые выходные данные |
|---|---|---|---|
| 2 | URL `"https://eda.menu"`, 10 страниц, 4 потока | 10 сохраненных HTML файлов | пройден |
| 3 | URL `"https://wrong.url"`, 5 страниц, 1 поток | Ошибка инициализации URL | пройден |
| 4 | URL `"https://eda.menu"`, 100 страниц, 16 потоков | 100 сохраненных HTML файлов | пройден |
| 5 | URL `"https://eda.menu"`, 10 страниц, 8 потоков | 10 сохраненных HTML файлов, время работы <5 секунд | пройден |
| 6 | URL `"https://eda.menu"`, 50 страниц, 32 потока | 50 сохраненных HTML файлов, время работы <10 секунд | пройден |
| 7 | URL `"https://eda.menu"`, 200 страниц, 64 потока | 200 сохраненных HTML файлов, время работы <30 секунд | пройден |
| 8 | URL `"https://eda.menu"`, 10 страниц, 256 потоков | 10 сохраненных HTML файлов, время работы <10 секунд | пройден |

# 5 Описание исследования

В ходе исследования оценивается производительность программы для

загрузки и обработки веб–страниц при различном количестве используемых потоков. Программа начинает с первой страницы, находит ссылки на страницы с рецептами и загружает их. Для каждого количества потоков проведены замеры времени при обработке до 10 страниц, содержащих информацию, относящуюся к рецептам (раздел ингредиентов). Каждый замер выполнен 20 раз, и результаты усреднены. Для извлечения контента использована библиотека curl, а замеры времени сохранены в CSV-файл. В итоговой таблице зафиксированы значения времени обработки страниц в зависимости от количества потоков.

## 5.1 Технические характеристики

Технические характеристики используемого устройства:

— операционная система: macOS Monterey;

— оперативная память: 8 ГБ;

— процессор: Quad-Core Intel Core i5 с тактовой частотой 1.4 ГГц;

## 5.2 Результаты исследования

В таблице 5.1 представлено время выполнения программы в зависимости от числа используемых потоков, измеренное в секундах.

Таблица 5.1 – Время выполнения программы в зависимости от количества потоков (в секундах)

| Количество потоков | Время выполнения |
|---:|---:|
| 1 | 2.81224 |
| 2 | 3.01663 |
| 4 | 2.82897 |
| 8 | 2.71858 |
| 16 | 2.38361 |
| 32 | 2.62481 |
| 64 | 2.60945 |
| 128 | 2.55341 |
| 256 | 3.01351 |

На рисунке 5.1 показано, как изменяется время выполнения при увеличении числа потоков, при постоянном количестве обрабатываемых страниц.

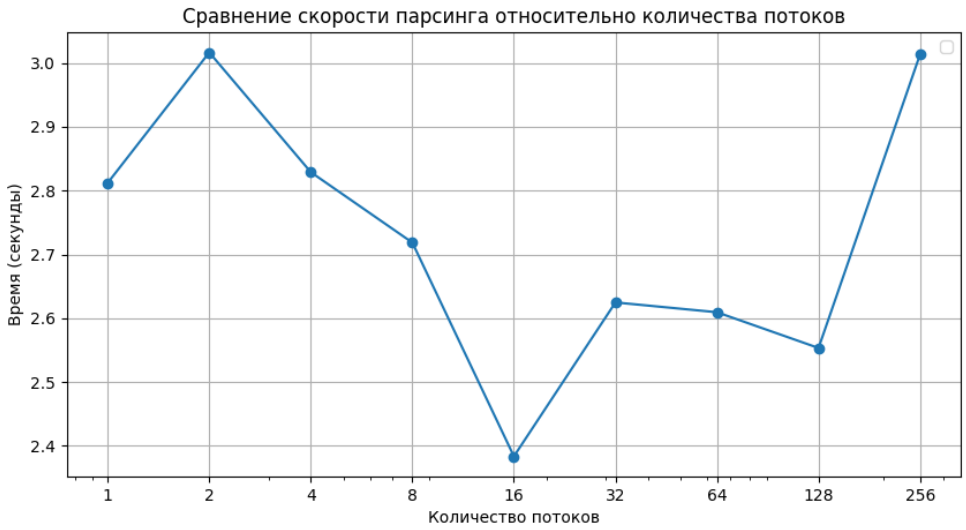


Рисунок 5.1 – Зависимость времени выполнения программы от количества потоков

## Вывод

Многопоточность значительно ускоряет выполнение программы по сравнению с однопоточной версией [3]. Время выполнения уменьшалось с увеличением числа потоков, но после определенного количества потоков улучшения производительности становились менее заметными. Время выполнения при использовании одного потока составило 2.81 секунды, в то время как при 16 потоках – 2.38 секунды, что указывает на сокращение времени выполнения на примерно 15%. Однако, при увеличении числа потоков с 16 до 128, улучшения в производительности были минимальными. Программа показывает наилучшую производительность при использовании 16 потоков, а наихудшую при использовании 2 и 256 потоков. При уменьшении числа потоков до 16 или 32 наблюдается заметное снижение эффективности работы. Таким образом, эффективность многопоточности ограничена и зависит от множества факторов, таких как производительность устройства и особенности задачи.

# ЗАКЛЮЧЕНИЕ

Цель работы достигнута. В ходе выполнения данной лабораторной работы решены следующие задачи:

— исследованы особенности предметной области и определены критерии для оптимизации алгоритмов;

— разработаны два алгоритма для обработки данных: один для последовательной обработки, второй — с использованием параллельных потоков;

— проведено тестирование разработанных алгоритмов с оценкой их эффективности по временным затратам и использованию памяти;

— подготовлены графики и таблицы, демонстрирующие результаты сравнительного анализа производительности алгоритмов;

— в отчете представлены полученные результаты о выполненной лабораторной работе.

Исследования показали, что использование параллельных потоков значительно ускоряет выполнение программы, особенно при увеличении количества потоков. Однако после достижения определенного числа потоков улучшения производительности становятся менее заметными, что связано с ограничениями устройства и особенностями решаемой задачи. Результаты исследования подтвердили эффективность многопоточности в обработке данных, при этом оптимальное количество потоков зависит от характеристик конкретной системы.

# СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ


1. *Танвар Ш.* Параллельное программирование на C# и .NET Core. — Москва : ДМК Пресс, 2022. — С. 272. — ISBN 978-5-97060-851-7.
2. EDA Menu [Электронный ресурс]. — Режим доступа: `https://eda.menu/` (дата обращения: 11.11.2024).
3. Threading in C# [Электронный ресурс]. — Режим доступа: `https://www.albahari.com/threading/` ; дата обращения: 2024-11-11.